

Stripping code blocks to produce a text-only version.

TP API REST - Gestion des étudiants

Présentation

Ce dépôt GitHub rassemble les fichiers produits pendant le TP de création d'une API REST pour une application de gestion d'étudiants.

Le but du projet est de développer la partie Back-end d'une application qui gère une liste d'étudiants stockée dans une base de données MySQL.

L'API propose plusieurs opérations sur les étudiants, grâce aux méthodes HTTP du protocole REST :

- **GET** : consulter les étudiants
- **POST** : ajouter un étudiant
- **PUT** : modifier un étudiant
- **DELETE** : supprimer un étudiant

Le projet a été construit étape par étape, en **quatre versions**, pour enrichir progressivement les fonctionnalités et renforcer la sécurité de l'API.

Technologies utilisées

- **Python** : langage de développement de l'API
- **Flask** : framework qui gère le serveur Web et les routes HTTP
- **MySQL** : système de gestion de base de données
- **mysql.connector** : bibliothèque Python de communication avec MySQL
- **Postman** : outil de test des requêtes HTTP
- **Visual Studio Code** : environnement de développement
- **GitHub** : plateforme de sauvegarde et de versionnement

Organisation du projet

Le dépôt « Création-d-une-API-Rest » contient :

- quatre fichiers d'API : `api_v1.py`, `api_v2.py`, `api_v3.py` et `api_v4.py` ;
- un fichier d'accès aux données : `db.py` ;
- un dossier Test regroupant `flask test.py`, `delete1.py`, `insert1.py` et `select_1.py`.

Description des fichiers

api_v1.py

Première version de l'API REST. Elle couvre les opérations CRUD de base sur la table `etudiant` : consulter, ajouter, modifier et supprimer un étudiant.

Les routes de cette version commencent par le préfixe `/v1/`. Elles permettent de lister tous les étudiants, d'en consulter un par son identifiant, d'en créer un, de le modifier et de le supprimer.

Cette version pose les fondations du fonctionnement de l'API.

api_v2.py

Deuxième version de l'API. Elle conserve les fonctionnalités de la v1 et y ajoute une gestion des erreurs plus solide. Des blocs `try / except` interceptent les problèmes qui peuvent survenir pendant l'exécution des requêtes.

Exemples de codes HTTP utilisés

- 200 : requête réussie
- 201 : ressource créée
- 400 : requête invalide
- 404 : identifiant inexistant
- 500 : erreur de connexion à la base de données

Par exemple, si l'on demande un étudiant dont l'identifiant n'existe pas, l'API renvoie une erreur 404.

Les routes reprennent les mêmes opérations que la v1, avec le préfixe `/v2/`.

api_v3.py

Troisième version de l'API. Elle apporte plusieurs évolutions importantes :

- mise en place d'un système d'authentification ;
- connexion par nom d'utilisateur et mot de passe ;
- création d'une table `user` dans la base de données ;
- ajout d'une route de connexion ;
- séparation des fonctions de requêtes SQL ;
- passage à une programmation modulaire ;
- passage à la programmation orientée objet avec la classe `Database`.

Authentification

Une nouvelle route de connexion (`/v3/login/`, en POST) permet à un utilisateur de se connecter. Il envoie son nom d'utilisateur et son mot de passe au format JSON. Si les identifiants sont corrects, l'API renvoie un message « Connexion OK » accompagné d'un token de session, avec le code HTTP 200 OK.

Ce token doit ensuite figurer dans l'en-tête **Authorization** de chaque requête vers une route protégée. Si les identifiants sont erronés, ou si le token est absent ou invalide, l'API répond 401 Unauthorized.

Routes CRUD de la v3

Les opérations sur les étudiants restent disponibles (lister, consulter, ajouter, modifier, supprimer), avec le préfixe **/v3/**, mais elles sont désormais protégées par le token.

api_v4.py

Quatrième version de l'API. Elle simplifie l'authentification en utilisant le mécanisme standard **HTTP Basic Auth**, à la place d'un token généré manuellement. Elle repose aussi sur une nouvelle version de la classe **Database** : chaque méthode ouvre sa propre connexion à la base de données et renvoie un **code numérique** (200/400/404/500), que l'API interprète ensuite.

Authentification

À chaque requête, l'authentification est contrôlée via **request.authorization** (en-tête HTTP Basic Auth, avec username et password), grâce à la méthode **db.login(request)**.

En cas d'échec, l'API répond par un message « Accès non autorisé » avec le code HTTP 401 Unauthorized.

Routes CRUD de la v4

Les mêmes opérations que dans les versions précédentes sont disponibles, avec le préfixe **/v4/**.

db.py

Le fichier **db.py** contient la classe **Database**. Elle rassemble toutes les fonctions qui envoient des requêtes à la base MySQL. Ce découpage sépare clairement la partie API de la partie accès aux données.

Depuis la v4, la classe ouvre une connexion à la demande à chaque appel, au lieu de maintenir une connexion unique ouverte en permanence.

Méthodes principales

Méthode	Rôle
connect()	Ouvre une connexion à la base de données MySQL
readAll()	Récupère tous les étudiants de la base de données

<code>readOne(id)</code>	Recherche un étudiant à partir de son identifiant
<code>create(...)</code>	Ajoute un nouvel étudiant dans la table <code>etudiant</code>
<code>update(...)</code>	Modifie les informations d'un étudiant existant
<code>delete(id)</code>	Supprime un étudiant
<code>login(request)</code>	Vérifie les identifiants HTTP Basic Auth à partir de la table <code>user</code>

Dossier Test

Le dossier `Test` regroupe les programmes Python écrits pour tester les opérations sur la base de données ainsi que l'utilisation de Flask.

Tests SQL

Ces tests vérifient les quatre opérations SQL principales servant à gérer les données : SELECT, INSERT, UPDATE et DELETE. Elles correspondent aux opérations CRUD utilisées par l'API.

1. SELECT

La commande SELECT sert à lire les informations d'une table MySQL. Elle permet d'afficher tous les étudiants ou de rechercher un étudiant à partir de son identifiant. Le fichier `select_1.py` sert aux tests de récupération des données.

2. INSERT

La commande INSERT ajoute une nouvelle ligne dans une table. Dans ce projet, elle permet d'ajouter un étudiant en renseignant son nom, son prénom, son e-mail et son téléphone. Un SELECT permet ensuite de vérifier l'ajout. Le fichier `insert1.py` sert aux tests d'ajout de données.

3. UPDATE

La commande UPDATE modifie les informations d'une ligne existante. La clause WHERE cible précisément l'étudiant à modifier : sans elle, la requête risquerait de modifier plusieurs lignes de la table. Un SELECT permet ensuite de vérifier la modification.

4. DELETE

La commande DELETE supprime une ligne d'une table. La clause WHERE désigne l'étudiant à supprimer. Il est essentiel de toujours poser une condition, sous peine d'effacer tous les étudiants de la table. Un SELECT permet ensuite de vérifier la suppression. Le fichier `delete1.py` sert aux tests de suppression de données.

Base de données

La base de données du projet s'appelle **ciel2027**.

Table etudiant

C'est la table principale de l'API. Elle contient les informations nécessaires à la gestion des étudiants :

- `idetudiant`
- `nom`
- `prenom`
- `email`
- `telephone`

Table user

Introduite dans la version 3, cette table enregistre les utilisateurs autorisés à s'authentifier auprès de l'API :

- `iduser`
- `username`
- `password`

Un utilisateur de test a été créé pour valider le fonctionnement de l'authentification : le nom d'utilisateur est « user1 » et le mot de passe « 123456 ».

Fonctionnement général de l'API

Le fonctionnement se fait en quatre étapes :

1. Le client (par exemple Postman) envoie une requête HTTP à l'API.
2. Flask, via le fichier de la version utilisée, reçoit la requête et détermine quelle fonction exécuter.
3. À partir de la version 3, les opérations sur la base de données sont assurées par les méthodes de la classe `Database`, définie dans `db.py`.
4. Cette classe dialogue avec le serveur MySQL (base ciel2027), et le résultat est renvoyé au client, le plus souvent sous forme d'objet JSON.

Exemple de fonctionnement (v4)

Pour obtenir la liste des étudiants avec la version 4, le client envoie une requête GET vers l'adresse locale du serveur (127.0.0.1, port 5000), sur la route `/v4/etudiants/`.

L'API contrôle d'abord l'authentification avec `db.login(request)`, puis appelle la méthode `readAll()`. Celle-ci exécute une requête SQL qui sélectionne tous les étudiants de la table. Les données récupérées sont converties en JSON puis renvoyées au client.

Tests depuis un autre poste

L'API a aussi été testée depuis un autre ordinateur du même réseau local.

Pour cela, le serveur Flask est configuré pour accepter les connexions venant des autres appareils du réseau : il écoute sur toutes les interfaces (adresse 0.0.0.0), sur le port 5000, avec le mode debug activé.

L'adresse IP locale du serveur utilisé pour les tests est 192.168.1.128. Depuis un autre ordinateur, on peut donc envoyer une requête vers cette adresse, sur le port 5000, par exemple sur la route `/v2/etudiants/`.

Ce test a confirmé que l'API était bien joignable depuis un autre poste du réseau local.

Codes HTTP utilisés

Code	Signification
200	Requête réussie
201	Ressource créée
400	Requête invalide
401	Accès non autorisé
404	Ressource inexistante
500	Erreur serveur ou erreur de connexion à la base de données

Versionnement de l'API

Le projet applique un système de versionnement : chaque évolution importante de l'API donne lieu à une nouvelle version.

Version 1 — v1

Elle met en place les opérations CRUD : GET, POST, PUT, DELETE.

Version 2 — v2

Elle ajoute surtout la gestion des erreurs (400 requête invalide, 404 ID inexistant, 500 erreur de base de données).

Version 3 — v3

Elle introduit :

- l'authentification des utilisateurs par token ;
- la route de connexion `/login/` ;
- la table `user` ;
- la programmation modulaire et orientée objet ;
- la classe `Database` et le fichier `db.py`.

Version 4 — v4

Elle fait évoluer l'authentification et l'architecture de `db.py` :

- authentification via **HTTP Basic Auth** à la place du token ;
- classe `Database` dont chaque méthode ouvre sa propre connexion à la demande ;
- méthodes `readAll`, `readOne`, `create`, `update`, `delete` et `login` qui renvoient des codes numériques interprétés par l'API.

Tableau récapitulatif des versions

Fonctionnalité	v1	v2	v3	v4
GET étudiants	✓	✓	✓	✓
GET étudiant par ID	✓	✓	✓	✓
POST étudiant	✓	✓	✓	✓
PUT étudiant	✓	✓	✓	✓
DELETE étudiant	✓	✓	✓	✓
Gestion des erreurs		✓	✓	✓
Gestion des utilisateurs			✓	✓

Authentification par token	✓	
Authentification HTTP Basic		✓
Classe Database	✓	✓
Fichier db.py	✓	✓
Programmation orientée objet	✓	✓

Requêtes HTTP de l'API (v4)

GET - Liste des étudiants

Une requête GET sur [/v4/etudiants/](#) retourne tous les étudiants. Elle exige une authentification HTTP Basic Auth valide. En cas de succès, la réponse est 200 OK.

GET - Un étudiant

Une requête GET sur [/v4/etudiants/](#) suivie de l'identifiant (par exemple 1) retourne l'étudiant correspondant. Si l'identifiant n'existe pas, la réponse est 404 Not Found.

POST - Ajouter un étudiant

Une requête POST sur [/v4/etudiants/](#) ajoute un étudiant. Ses informations (nom, prénom, e-mail, téléphone) sont transmises dans le corps de la requête, au format JSON. En cas de succès, la réponse est 201 Created.

PUT - Modifier un étudiant

Une requête PUT sur [/v4/etudiants/](#) suivie de l'identifiant (par exemple 1) modifie l'étudiant correspondant. Les nouvelles informations (nom, prénom, e-mail, téléphone) sont transmises au format JSON. En cas de succès, la réponse est 200 OK. Si l'étudiant n'existe pas, la réponse est 404 Not Found.

DELETE - Supprimer un étudiant

Une requête DELETE sur [/v4/etudiants/](#) suivie de l'identifiant (par exemple 1) supprime l'étudiant correspondant. En cas de succès, la réponse est 200 OK. Si l'étudiant n'existe pas, la réponse est 404 Not Found.

Authentification (v4)

Toutes les routes ci-dessus demandent une authentification **HTTP Basic Auth** (username et password issus de la table `user`), contrôlée par `db.login(request)`. Si les identifiants sont incorrects ou absents, la réponse est 401 Unauthorized.

Tests Postman

Postman sert à vérifier que les différentes routes de l'API fonctionnent correctement, pour les quatre versions. Les tests de la v4 comprennent notamment :

- la liste de tous les étudiants ;
- la consultation d'un étudiant existant (id 1) et d'un étudiant inexistant (id 999) ;
- l'ajout d'un étudiant ;
- la modification d'un étudiant existant et d'un étudiant inexistant ;
- la suppression d'un étudiant existant et d'un étudiant inexistant.

Ces tests couvrent aussi bien les cas normaux que les cas d'erreur (id inexistant, requête invalide, absence d'authentification).

Une collection Postman est enregistrée et contient la v1 jusqu'à la v4

Cybersécurité et améliorations possibles

L'API fonctionne correctement dans le cadre du TP, mais plusieurs améliorations seraient nécessaires pour un usage réel.

Protection des mots de passe

Dans une vraie application, les mots de passe ne doivent pas être stockés en clair dans la base. Il vaudrait mieux les hacher de façon sécurisée, à l'aide d'une bibliothèque adaptée.

Protection contre les injections SQL

Dans le TP, les requêtes SQL sont construites avec des f-strings, comme l'imposaient les consignes. En production, il faudrait plutôt utiliser des requêtes préparées pour réduire les risques d'injection SQL.

Utilisation du HTTPS

L'API pourrait être servie en HTTPS afin de chiffrer les échanges entre le client et le serveur, ce qui protégerait notamment les informations sensibles envoyées lors de l'authentification.

Authentification améliorée

Le système de connexion pourrait aussi évoluer avec :

- des tokens JWT dotés d'une durée d'expiration des sessions ;

- une limite sur le nombre de tentatives de connexion ;
- une gestion plus fine des droits des utilisateurs.

Protection contre les attaques XSS

Il serait également utile de valider et de vérifier les données reçues, pour limiter les risques d'injection de contenu malveillant.

J'ai remplacé tous les blocs de code (arborescence, routes, JSON, SQL, Python, schéma) par des descriptions en texte. Seuls les noms de fichiers, de méthodes et de tables sont restés entre accents graves, car ils sont nécessaires à la compréhension. Voulez-vous que j'en fasse un fichier `README.md` ou un document Word ?